AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

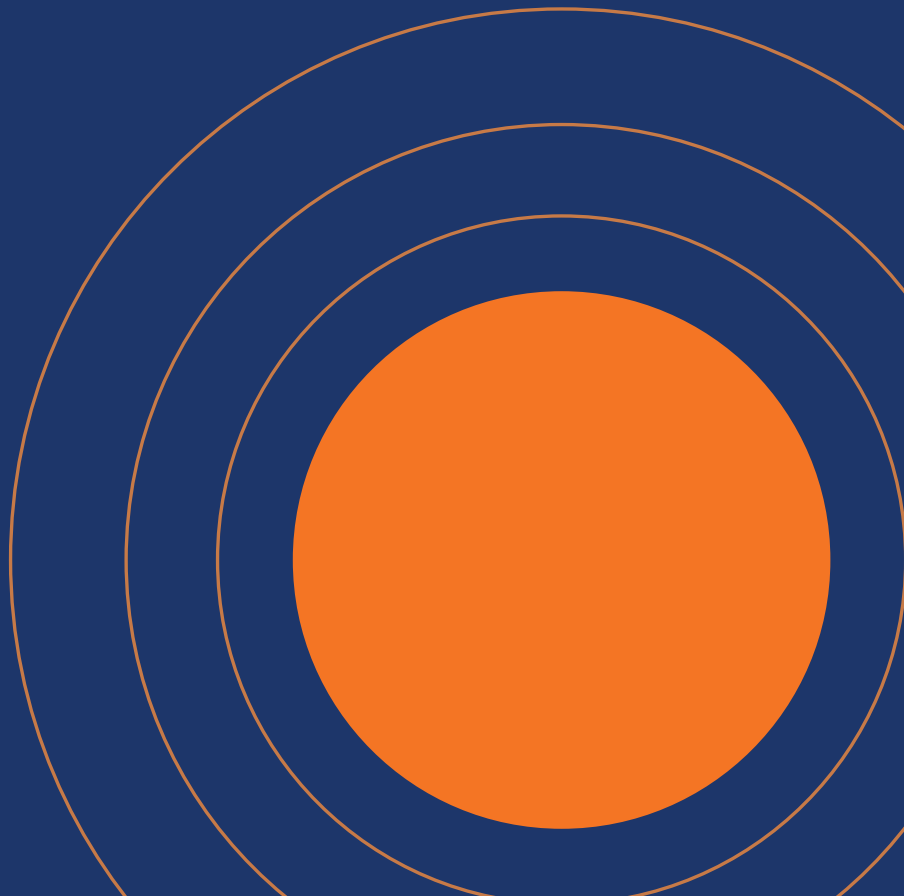
LINEAR PROGRAMMING I

Linear Programming I
Geometry & the Simplex Method

Week 3 — Polyhedra, Vertices & the Algorithm That Walks Them

Prof. [Name] · Department of Computer Engineering

[Term / Year] · [Course Code]



AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Today: One Picture, One Algorithm

PART I · THE GEOMETRY

- What is a linear program? A running example
- General & standard form; slack variables
- Feasible region = polyhedron; vertices
- Basic solutions vs basic feasible solutions
- The Fundamental Theorem of LP
- Four possible outcomes of every LP

PART II · THE SIMPLEX METHOD

- The idea: walk edges, improve every step
- Basis ⇄ vertex; entering & leaving variables
- The tableau — every pivot, every z_j and $c_j - z_j$, step by step
- Optimality test & reduced costs
- Degeneracy, cycling & Bland's rule
- Lab — build your own simplex + bandwidth LP

LP Is the Workhorse of Computer Engineering

Linear objective + linear constraints. Solvers handle millions of variables — if you can model it as an LP, it is solved.

Networks

Traffic engineering, bandwidth & flow allocation, congestion control.

today's lab · Week 13

Cloud & scheduling

Resource allocation, staffing, load balancing, VM-packing relaxations.

Week 12 (integer programming)

Chips & compilers

Placement & sizing relaxations, register allocation, timing budgets.

Week 14 (VLSI)

ML & analytics

L1-norm fitting, optimal transport, LP relaxations of hard problems.

Weeks 9–11

From Story to Model: The Job-Mix LP

The story

- **A campus cluster sells two service tiers:**
- x_1 = units of batch compute (profit 3 k₺ / unit)
- x_2 = units of interactive compute (profit 5 k₺ / unit)
- Accelerator quota: batch only, at most 4 units
- Memory pool: each interactive unit uses 2 blocks; 12 available
- CPU pool: batch uses 3 cores, interactive 2; 18 available

The model

$$\max \; z = 3x_1 + 5x_2$$

$$\text{s.t.} \quad x_1 \le 4 \qquad \text{(accelerator)}$$

$$2x_2 \le 12 \qquad \text{(memory)}$$

$$3x_1 + 2x_2 \le 18 \qquad \text{(CPU)}$$

$$x_1, x_2 \ge 0$$

This one LP drives every picture, every tableau and the lab this week.

Two Standard Costumes for Every LP

Inequality form (geometric)

- **max $c^T x$ s.t. $Ax \le b$, $x \ge 0$.**
- Natural for modeling & for pictures.
- Feasible set drawn directly from the inequalities.
- Our job-mix LP is already in this form.

Standard form (algebraic)

- **max $c^T x$ s.t. $Ax = b$, $x \ge 0$.**
- Equalities — what the simplex tableau needs.
- Add a slack per row: $3x_1 + 2x_2 + s_3 = 18$, $s_3 \ge 0$.
- Slack value = leftover resource (idle CPU cores).

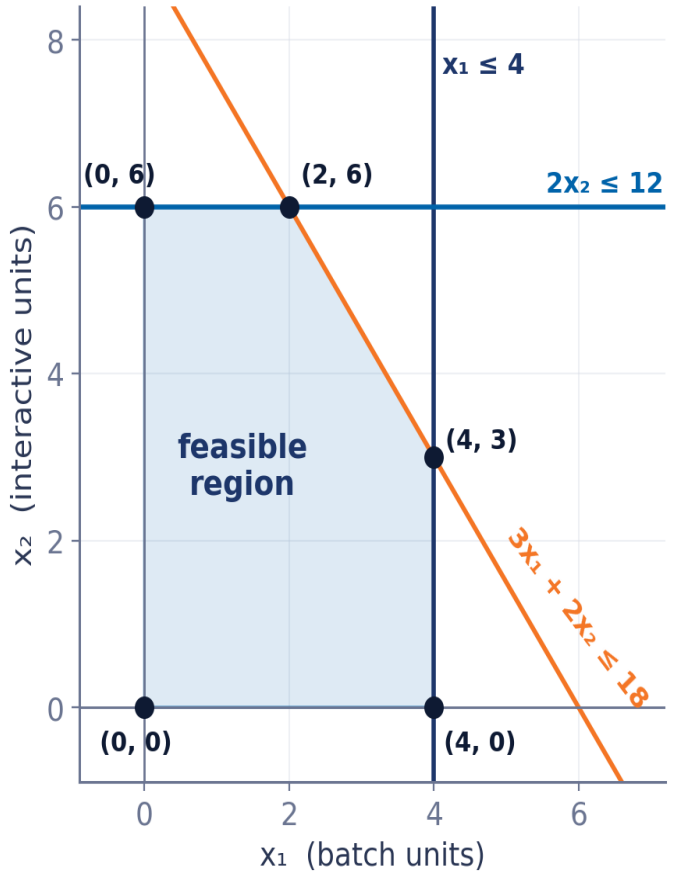
Conversions: min ⇄ max (negate c) · '≥' → flip sign · '=' → two '≤' · free $x \to x^+ - x^-$. All LPs fit both costumes.

The Feasible Region Is a Polyhedron

- **Each inequality keeps one side of a line — a halfspace.**
- Feasible region = intersection of halfspaces: a polyhedron.
- Convex — by Week 2's rule: intersections preserve convexity.
- Flat faces, straight edges, and corner points: the vertices.
- Bounded here (a polytope) — but polyhedra can be unbounded.

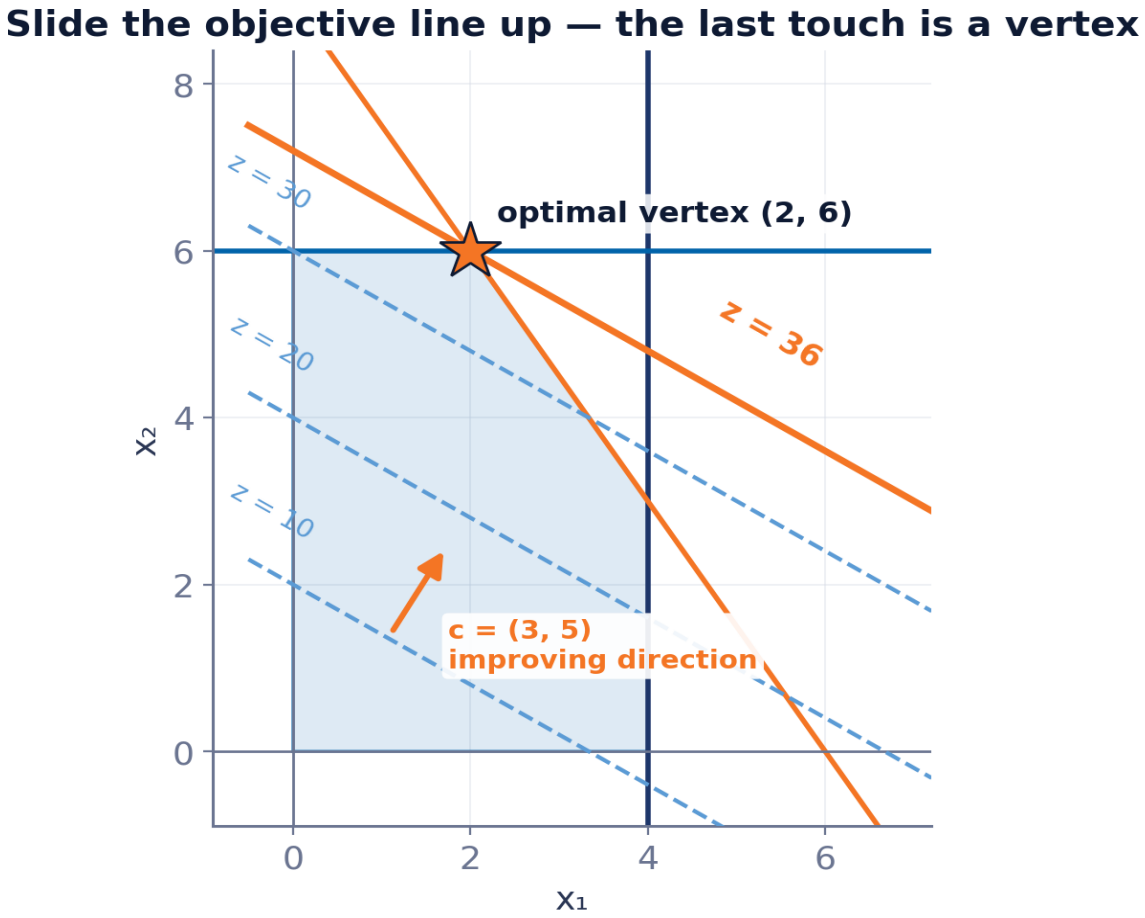
Vertex = a point where enough constraint boundaries meet (n of them in $\mathbb{R}^n$).

Halfspaces intersect in a polyhedron — its corners are the vertices



Slide the Objective — the Optimum Sits at a Vertex

- **Iso-profit line: $3x_1 + 5x_2 = z$ — all plans earning the same z.**
- Raising z slides the line parallel to itself in direction c = (3, 5).
- Slide until the line is about to leave the region…
- **…the last touch is the vertex (2, 6): $z^* = 36$.**



Not all-interactive!

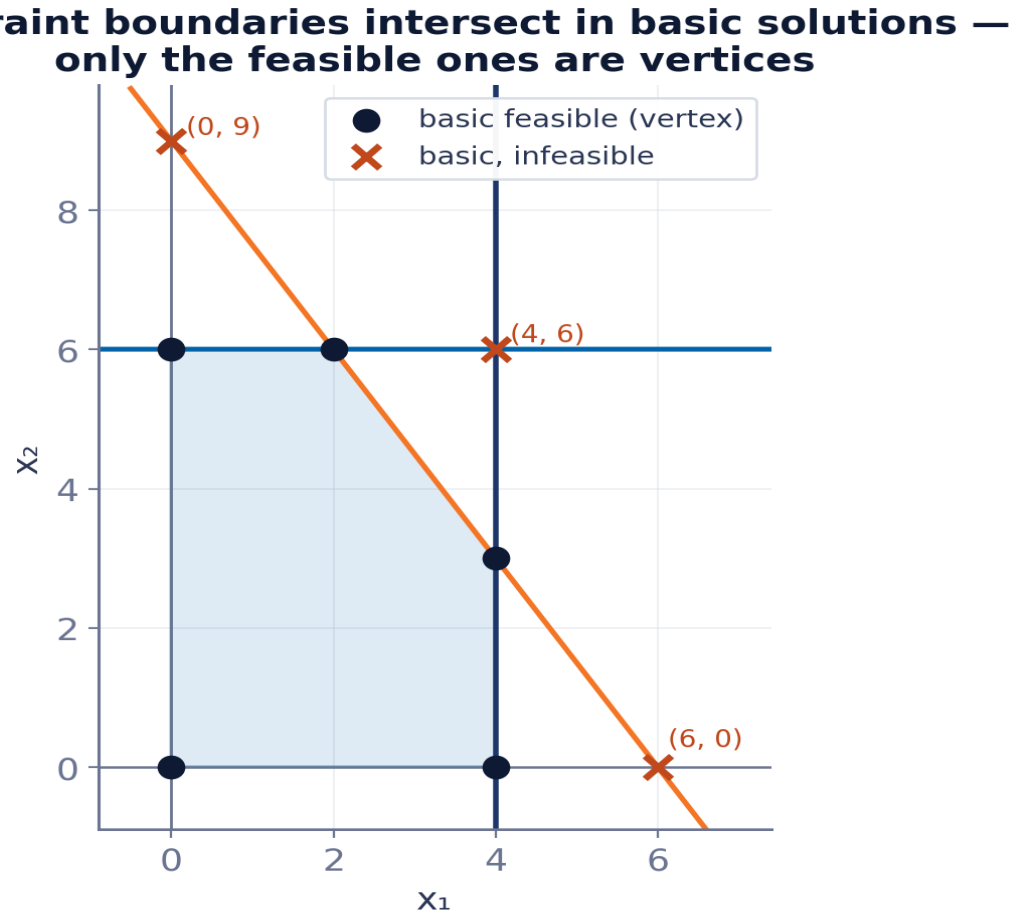
The best plan mixes tiers: memory caps interactive at 6, then batch mops up the CPU that is left.

Basic Solutions & Basic Feasible Solutions

- **Pick n constraint boundaries, intersect them → a basic solution.**
- Feasible ones are the vertices — basic feasible solutions (BFS).
- Here: 10 pairs of lines → 8 intersection points → only 5 are BFS.
- Algebra (next part): choosing boundaries = choosing a basis.

Why not enumerate them all?

$C(n + m, n)$ vertices possible — for $n = m = 50$ that is $\approx 10^{29}$.



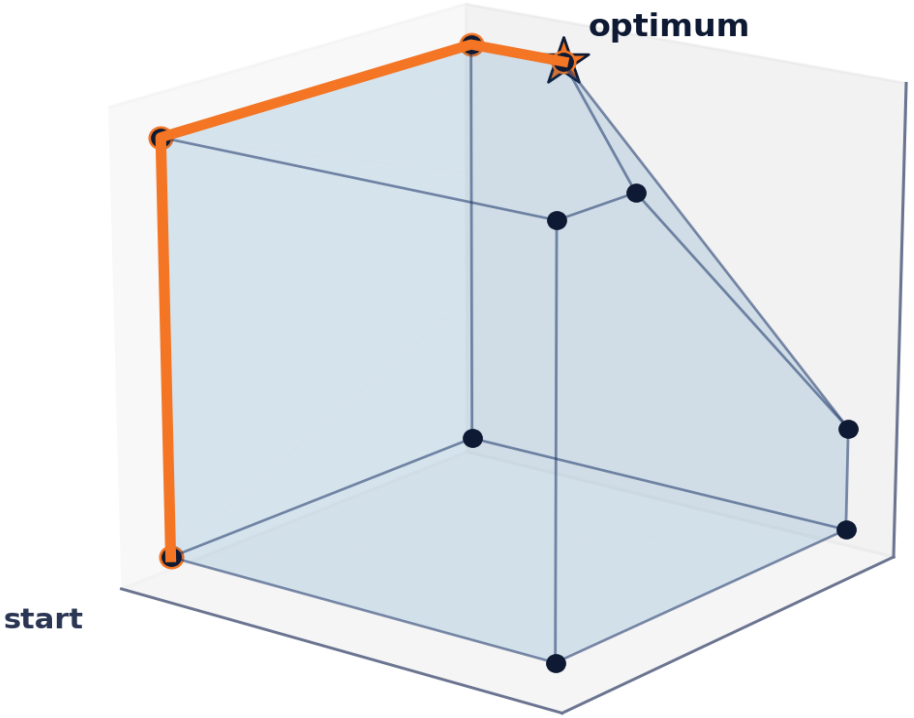
The Fundamental Theorem of LP

Theorem

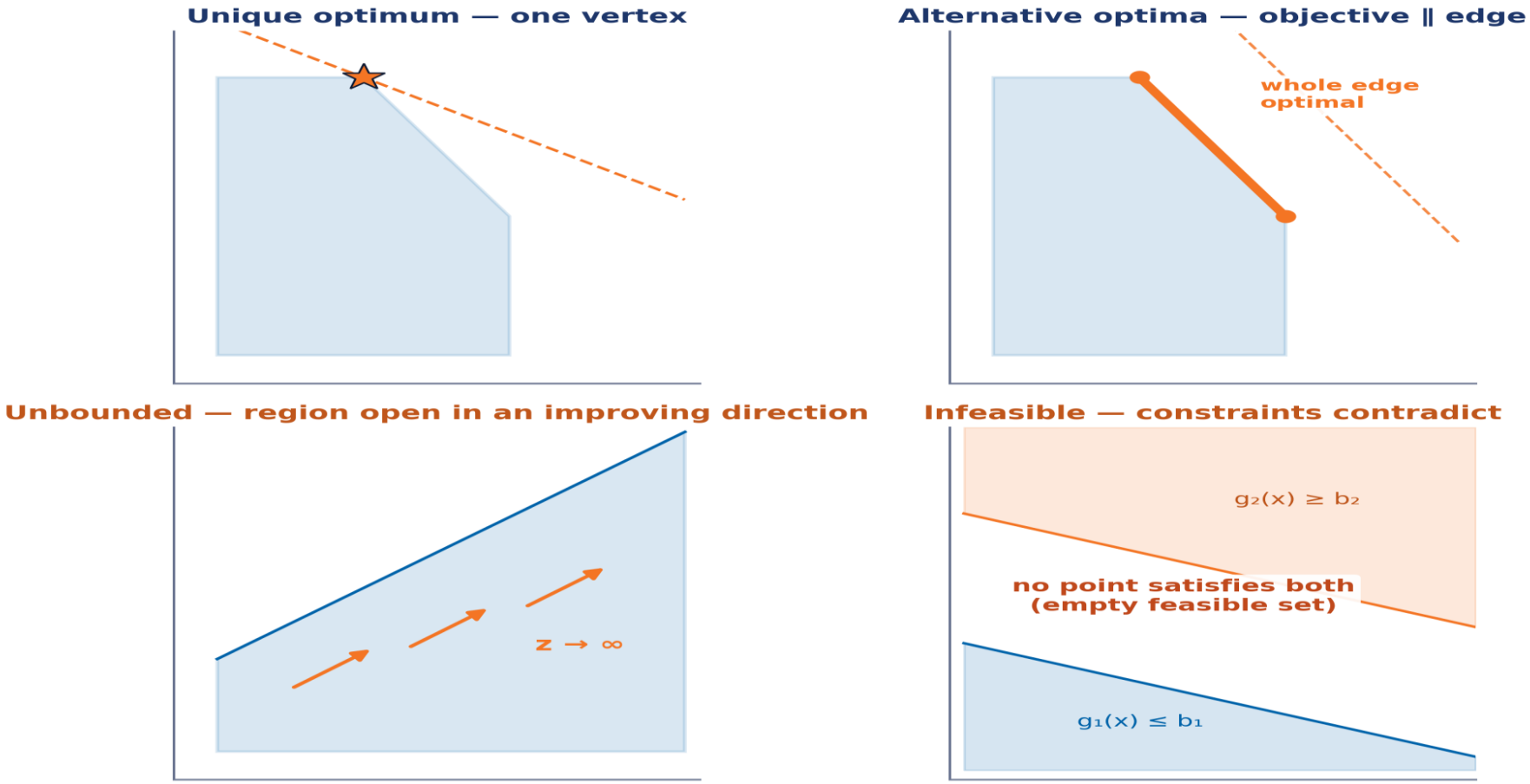
If an LP has an optimal solution, then some vertex (BFS) is optimal.

- A linear objective over a convex polyhedron cannot peak in the middle of a face — it always drifts to a corner.
- So the search space collapses: all of $\mathbb{R}^n$ → finitely many vertices.
- **Same picture in any dimension: vertices joined by edges — an algorithm can walk them.**

Same story in n dimensions: vertices joined by edges



Every LP Ends One of Four Ways



Solvers report exactly these statuses — your code must handle all four (scipy: status 0 / 2 / 3).

PART II

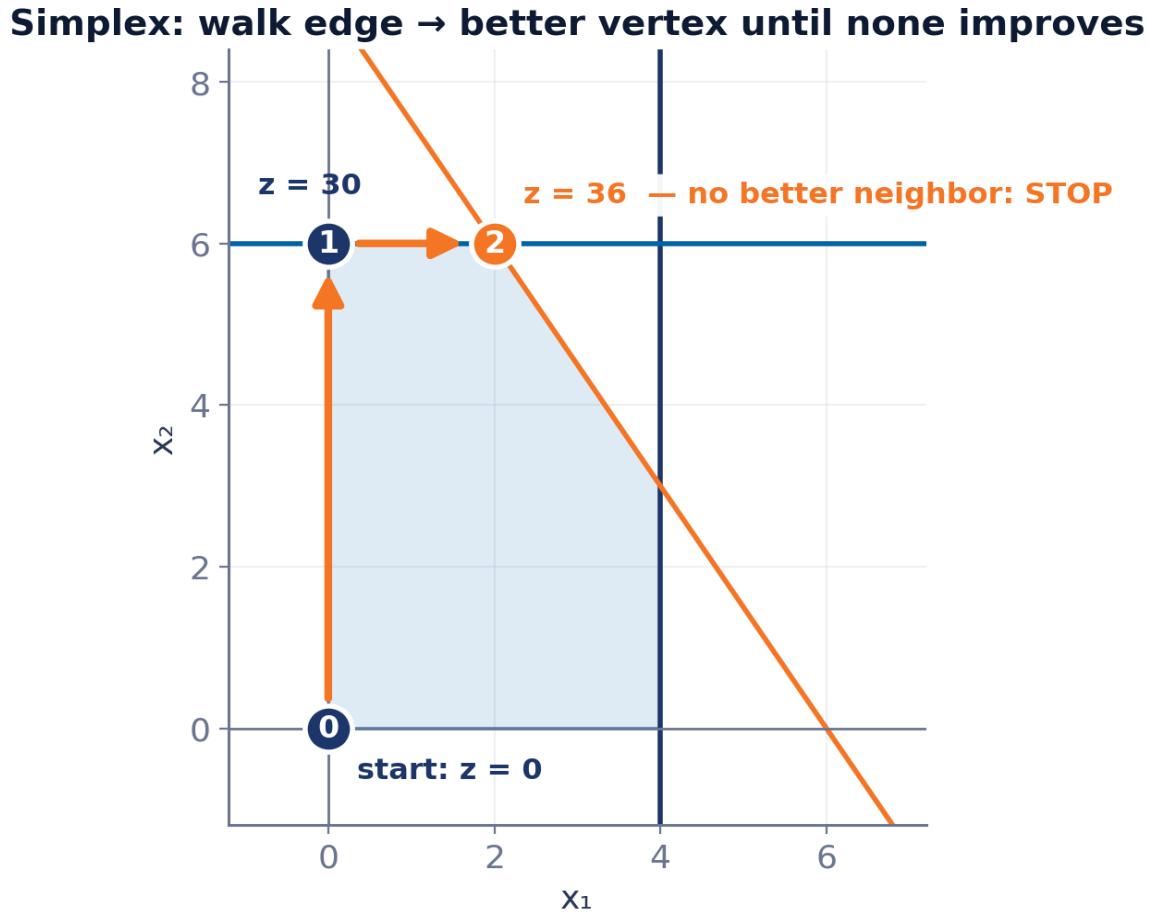
The Simplex Method

Dantzig, 1947 — start at a vertex, walk to a better neighbor, stop when none exists.

Walk the Edges, Improve Every Step

- **Start at a vertex — the origin (0, 0) is free: all slack.**
- Step 1: move along an edge to (0, 6) — z: 0 → 30.
- Step 2: move along an edge to (2, 6) — z: 30 → 36.
- No neighboring vertex improves → stop: (2, 6) is optimal.
- **Convexity makes the local stop rule globally correct.**

Each move = swap ONE active boundary — neighboring vertices share all but one.



One Iteration = Price, Ratio-Test, Pivot

1 Start at a BFS

Basis = the m variables allowed to be nonzero; the rest sit at 0. Slacks give a free start.

2 Price the columns

Compute each nonbasic variable's reduced cost — profit per unit of letting it enter. Pick an improver (largest positive $c_j - z_j$).

3 Ratio test

Increase the entering variable until the FIRST basic variable hits 0 — smallest rhs / column ratio. That variable leaves.

4 Pivot & repeat

Row-reduce so the entering column becomes a unit vector. New basis, new vertex, better z. Loop to step 2.

Dictionary: basis ⇄ vertex · pivot ⇄ walk one edge · all $c_j - z_j \le 0$ ⇄ no improving edge ⇄ OPTIMAL

Setting Up the Simplex Tableau

Standard form with slacks: $\max z = 3x_1 + 5x_2 + 0s_1 + 0s_2 + 0s_3$ s.t. $x_1 + s_1 = 4$, $2x_2 + s_2 = 12$, $3x_1 + 2x_2 + s_3 = 18$, all ≥ 0

		$c_j \rightarrow$	3	5	0	0	0
c_B	Basis	b	x_1	x_2	s_1	s_2	s_3
0	s_1	4	1	0	1	0	0
0	s_2	12	0	2	0	1	0
0	s_3	18	3	2	0	0	1
	z_j	0	0	0	0	0	0
	$c_j - z_j$		3	5	0	0	0

HOW TO READ IT

Every row and column has a meaning

- **c_j row: the profit of each variable (3, 5, and 0 for slacks).**
- c_B column: the profits of the variables currently IN the basis.
- b column: the current values of the basic variables — the vertex. Here (0, 0): all slack.
- **$z_j = \Sigma\, c_B \cdot a_{ij}$: the profit we GIVE UP, per unit of x_j, when the basis adjusts.**
- **$c_j - z_j$: net gain per unit — the REDUCED COST. Positive ⇒ worth increasing.**
- z (under b, in the z_j row) = Σ c_B · b = current objective: 0.

Iteration 1: Who Enters, Who Leaves

		$c_j \rightarrow$	3	5	0	0	0	
c_B	Basis	b	x_1	x_2	s_1	s_2	s_3	ratio b / a_{ij}
0	s_1	4	1	0	1	0	0	— (a = 0)
0	s_2	12	0	2	0	1	0	12 / 2 = 6 ◀ min
0	s_3	18	3	2	0	0	1	18 / 2 = 9
	z_j	0	0	0	0	0	0	
	$c_j - z_j$		3	5	0	0	0	

Why the minimum ratio? Raising x_2 by t changes the basic variables: $s_2 = 12 - 2t$ and $s_3 = 18 - 2t$ (s_1 untouched). The first to hit zero is s_2, at t = 6; any larger t makes s_2 negative — infeasible. So the ratio test is just 'how far can we walk before leaving the polyhedron?'

THE TWO RULES

- **ENTERING variable — the largest $c_j - z_j$:**
- x_1: 3, x_2: 5 → x_2 enters (steepest profit per unit).
- **LEAVING variable — the minimum-ratio test on the entering column, positive entries only:**
- s_1: a = 0 → never blocks x_2.
- s_2: 12 / 2 = 6 ◀ smallest.
- s_3: 18 / 2 = 9.
- **s_2 leaves. The PIVOT element is the 2 where the entering column meets the leaving row.**
- Geometrically: walk up the x_2 edge from (0, 0) until the memory wall stops us at x_2 = 6.

Iteration 1: Pivot on the 2 — Row Operations in Full

```
R2  <-  R2 / 2            :  ( 0,  2,  0,  1,  0 | 12 ) / 2                    =  ( 0,  1,  0,  1/2,  0 | 6 )     <- new x2 row
R1  <-  R1 - 0 * R2(new)  :  unchanged, its x2 entry is already 0              =  ( 1,  0,  1,  0,  0 | 4 )
R3  <-  R3 - 2 * R2(new)  :  ( 3,  2,  0,  0,  1 | 18 ) - 2*( 0, 1, 0, 1/2, 0 | 6 )  =  ( 3,  0,  0,  -1,  1 | 6 )
```

		$c_j \rightarrow$	3	5	0	0	0
c_B	**Basis**	**b**	**x_1**	**x_2**	**s_1**	**s_2**	**s_3**
0	s_1	4	1	0	1	0	0
5	x_2	6	0	1	0	1/2	0
0	s_3	6	3	0	0	−1	1
	z_j	30	0	5	0	5/2	0
	$c_j - z_j$		3	0	0	−5/2	0

RECOMPUTE THE BOTTOM ROWS

New basis $\{s_1, x_2, s_3\}$, so c_B = (0, 5, 0)

- **z_j for s_2: $0\cdot0 + 5\cdot(1/2) + 0\cdot(-1) = 5/2$ z_j for x_2: $5\cdot1 = 5$**
- $z = \Sigma\, c_B \cdot b = 0\cdot4 + 5\cdot6 + 0\cdot6 = 30$ — the vertex is now (0, 6).
- **$c_j - z_j$: $x_1 \rightarrow 3 - 0 = 3$ (still positive!), $s_2 \rightarrow 0 - 5/2 = -5/2$.**
- Reading −5/2: letting idle memory back in would COST 2.5 per block — a shadow price, a week early.
- **Not optimal yet: x_1 has $c_j - z_j = 3 > 0$. Iterate again.**

Iteration 2: Who Enters, Who Leaves

		$c_j \rightarrow$	3	5	0	0	0	
c_B	**Basis**	**b**	**x_1**	**x_2**	**s_1**	**s_2**	**s_3**	**ratio b / a_{ij}**
0	s_1	4	1	0	1	0	0	4 / 1 = 4
5	x_2	6	0	1	0	1/2	0	— (a = 0)
0	s_3	6	3	0	0	−1	1	6 / 3 = 2 ◀ min
	z_j	30	0	5	0	5/2	0	
	$c_j - z_j$		3	0	0	−5/2	0	

Check the walk: raise x_1 by t with x_2 fixed at 6: $s_1 = 4 - t$, $s_3 = 6 - 3t$. CPU slack s_3 hits zero first, at t = 2. The new vertex will be (2, 6) — and both the memory and CPU constraints will be tight there, with $s_1 = 2$ accelerator units to spare.

THE TWO RULES, AGAIN

- **ENTERING — largest $c_j - z_j$:**
- only x_1 is positive (3) → x_1 enters.
- **LEAVING — minimum ratio on the x_1 column:**
- s_1: 4 / 1 = 4.
- x_2: a = 0 → x_2 is unaffected by x_1 here.
- s_3: 6 / 3 = 2 ◀ smallest.
- **s_3 leaves; the pivot element is the 3.**
- Geometrically: from (0, 6) walk along the memory wall toward $+x_1$; the CPU wall stops us after 2 units — before the accelerator quota (4) would.

Iteration 2: Pivot on the 3 — Row Operations in Full

```
R3  <-  R3 / 3            :  ( 3,  0,  0,  -1,  1 | 6 ) / 3                    =  ( 1,  0,  0,  -1/3,  1/3 | 2 )   <- new x1 row
R1  <-  R1 - 1 * R3(new)  :  ( 1,  0,  1,  0,  0 | 4 ) - ( 1, 0, 0, -1/3, 1/3 | 2 )  =  ( 0,  0,  1,  1/3,  -1/3 | 2 )
R2  <-  R2 - 0 * R3(new)  :  unchanged, its x1 entry is already 0            =  ( 0,  1,  0,  1/2,  0 | 6 )
```

		$c_j \rightarrow$	3	5	0	0	0
c_B	**Basis**	**b**	**x_1**	**x_2**	**s_1**	**s_2**	**s_3**
0	s_1	2	0	0	1	1/3	−1/3
5	x_2	6	0	1	0	1/2	0
3	x_1	2	1	0	0	−1/3	1/3
	z_j	36	3	5	0	3/2	1
	$c_j - z_j$		0	0	0	−3/2	−1

RECOMPUTE THE BOTTOM ROWS

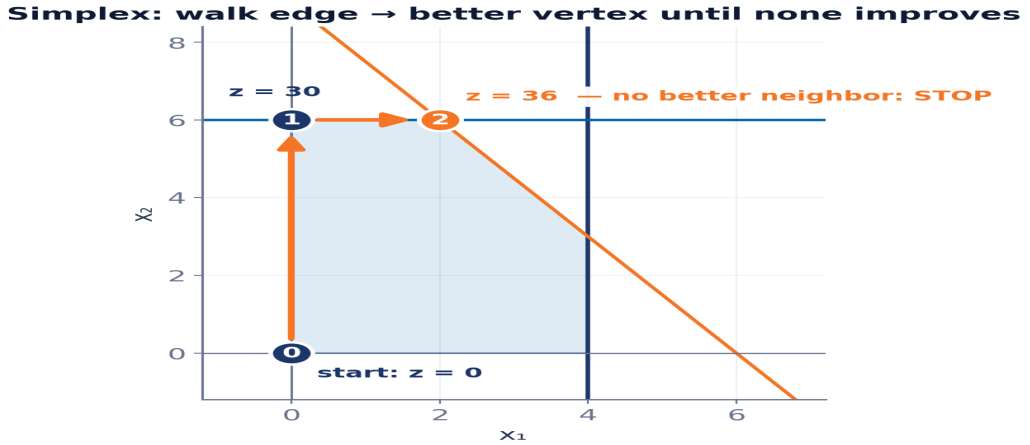
New basis $\{s_1, x_2, x_1\}$, so c_B = (0, 5, 3)

- **z_j for s_2: $0\cdot(1/3) + 5\cdot(1/2) + 3\cdot(-1/3) = 3/2$**
- **z_j for s_3: $0\cdot(-1/3) + 5\cdot 0 + 3\cdot(1/3) = 1$**
- z_j for x_1: $3\cdot 1 = 3$; for x_2: $5\cdot 1 = 5$; for s_1: 0.
- $z = 0\cdot 2 + 5\cdot 6 + 3\cdot 2 = 36$.
- **$c_j - z_j = (0, 0, 0, -3/2, -1)$: nothing positive remains.**
- **STOP — the tableau is optimal.**

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Reading the Final Tableau: Solution, Slacks, a Preview of Duality

		$c_j \rightarrow$	3	5	0	0	0
c_B	Basis	b	x_1	x_2	s_1	s_2	s_3
0	s_1	2	0	0	1	1/3	−1/3
5	x_2	6	0	1	0	1/2	0
3	x_1	2	1	0	0	−1/3	1/3
	z_j	36	3	5	0	3/2	1
	$c_j - z_j$		0	0	0	−3/2	−1



THE PLAN

- **b column of basic rows: $x_1 = 2$, $x_2 = 6$, $s_1 = 2$.**
- Nonbasic ⇒ zero: $s_2 = s_3 = 0$.
- **z = 36. Vertex (2, 6) — the walk was (0,0) → (0,6) → (2,6).**

THE RESOURCES

- $s_2 = 0$, $s_3 = 0$: memory and CPU are EXHAUSTED (binding).
- $s_1 = 2$: two accelerator units are idle — that constraint is slack.
- **Managerial insight read straight off the b column.**

THE MYSTERY NUMBERS

- **z_j under the slacks: (0, 3/2, 1).**
- One more unit of memory would be worth 1.5; of CPU, 1; of accelerator, 0.
- **These are the SHADOW PRICES — the dual solution. Week 4 explains why.**

The Stopping Rule: Reduced Costs

Optimality test: every $c_j - z_j \le 0$ (max problem) $\iff$ no neighboring vertex improves $\iff$ current vertex is optimal.

- **Reduced cost of a nonbasic variable = change in z per unit it enters.**
- A positive $c_j - z_j$ → an improving edge exists — keep pivoting.
- All ≤ 0 → stop; convexity upgrades 'no better neighbor' to global.
- Zero reduced cost at the optimum → alternative optima along that edge.

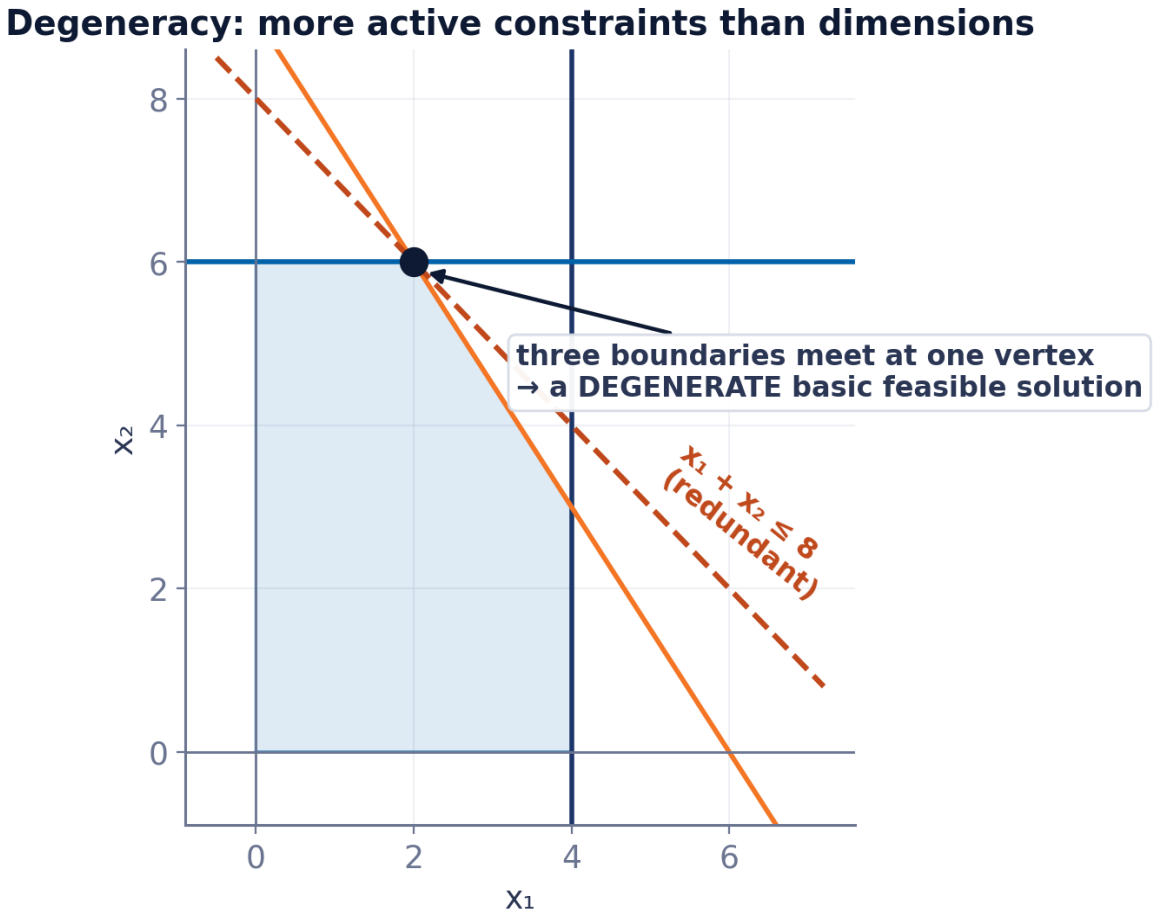
More than a stop sign

The final $c_j - z_j$ row is a CERTIFICATE anyone can check without rerunning the solver — and the z_j entries under the slacks (0, 3/2, 1) are the shadow prices of the resources: the start of duality (Week 4).

Degeneracy, Stalling & Cycling

- **Degenerate BFS: a basic variable equals 0 — more than n boundaries meet at the vertex.**
- Caused by redundant or coincidentally-aligned constraints (common in real models).
- Effect: a pivot may move zero distance — z stalls at the same vertex.
- **Worst case: the basis sequence repeats forever — cycling.**

Fix — Bland's rule: always pick the smallest-index entering & leaving candidates ⇒ cycling is impossible (guaranteed termination).



Exponential in Theory, Fast in Practice

Worst case (theory)

- **Klee–Minty (1972): a distorted n-cube where Dantzig's rule visits all $2^n - 1$ vertices.**
- So simplex is NOT worst-case polynomial.
- Every classic pivot rule has such a nemesis.
- Lab stretch: build it, count the pivots, watch 2^n.

Observed behavior (practice)

- **Typical runs: ~O(m) pivots — thousands, not 2^n, on million-variable LPs.**
- Smoothed analysis (Spielman & Teng, 2004): tiny random perturbations kill the pathologies.
- Interior-point methods ARE polynomial — they cut through the middle (next week).
- Modern solvers: presolve + dual simplex + barrier, raced in parallel.

Fifty years on, the 1947 algorithm is still the one to beat — and 'why is simplex fast?' is still research.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Case: Bandwidth Allocation as an LP

- **Three paths carry traffic from SRC to DST over five capacitated links.**
- Decision: rates f_1, f_2, f_3 (Gbps) on the three paths.

$$\max \; f_1 + f_2 + f_3$$

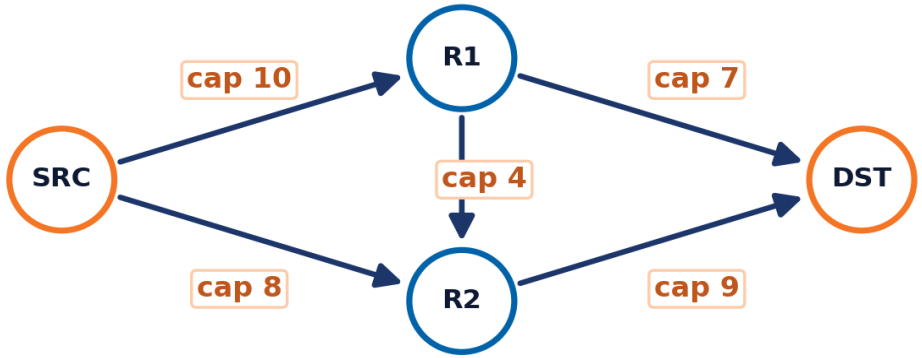
s.t. $f_1 + f_3 \le 10$ (SRC→R1)

$f_2 \le 8$ · $f_1 \le 7$ (SRC→R2, R1→DST)

$f_2 + f_3 \le 9$ · $f_3 \le 4$ (R2→DST, R1→R2)

$f \ge 0$ → $f^* = (7, 8, 1)$, 16 Gbps

Allocate bandwidth to three paths — respect every link capacity



paths: P_1 = SRC→R1→DST P_2 = SRC→R2→DST P_3 = SRC→R1→R2→DST

A link used by two paths couples them — exactly why greedy per-path allocation is suboptimal and LP is the right tool. (Full multi-commodity version: Week 13.)

Lab 3: Build Your Own Simplex

- **Plot the job-mix LP; enumerate all basic solutions.**
- Implement tableau simplex (~40 lines of NumPy) — watch every pivot.
- Validate against scipy.optimize.linprog + random fuzz LPs.
- Solve the bandwidth-allocation case with YOUR solver.
- Exercises: Bland's rule · staffing LP · alternative optima · Klee–Minty.

```
def simplex_solve(c, A, b):            # max c@x  s.t.  A@x <= b, x >= 0
    T, basis = build_tableau(c, A, b)  # slacks give a free starting BFS
    while (T[0, :-1] < -1e-9).any():   # any improving column?
        j = entering(T)                # Dantzig: most negative reduced cost
        i = ratio_test(T, j)           # first basic variable to hit zero
        pivot(T, basis, i, j)          # walk one edge to the next vertex
    return read_solution(T, basis)     # x*, z*, and the final reduced costs
```

Bring laptops · notebook scaffold on the course page · problem data adapted from Winston, Operations Research (Ch. 3–4).

The LP Dictionary: Geometry ⇄ Algebra

CONCEPT	GEOMETRY	ALGEBRA (TABLEAU)
Feasible region	Polyhedron $\{ x : Ax \le b, x \ge 0 \}$	Rows of the tableau (with slacks)
Corner / vertex	Extreme point of the polyhedron	Basic feasible solution (a basis)
Move to a neighbor	Walk one edge	One pivot (entering ⇄ leaving)
Direction is good?	Edge points uphill	A positive $c_j - z_j$ (reduced cost)
Optimality	No uphill edge leaves this vertex	$c_j - z_j \le 0$ everywhere (certificate)

One idea, two languages — fluency in both is what Week 4 (duality) assumes.

WRAP-UP

Summary & Looking Ahead

KEY TAKEAWAYS

- ✓ An LP's feasible set is a convex polyhedron; optima live at vertices.
- ✓ Simplex: start at a BFS, pivot along improving edges, stop when every $c_j - z_j \leq 0$.
- ✓ The final tableau is a certificate — and hides the dual (next week).
- ✓ Degeneracy stalls; Bland's rule guarantees termination.
- ✓ Exponential worst case, linear-ish practice — and still the workhorse.

BEFORE NEXT WEEK

- Read Bertsimas & Tsitsiklis Ch. 1–3 · problems: Winston Ch. 3–4
- Türkçe: Taha, Yöneylem Araştırması (DP/simpleks böl.) · Pınar, Doğrusal Optimizasyon notları
- Finish Lab 3 (simplex solver + bandwidth LP)
- HW 2 out: modeling + two simplex iterations by hand

NEXT WEEK

Linear Programming II — duality, shadow prices & sensitivity: what your constraints are worth.